

ĐỒ ÁN THIẾT KẾ VI MẠCH SỐ

THIẾT KẾ VÀ ĐÁNH GIÁ HỆ THỐNG PicoRV32 + DMA + DMA-SIDE IOMMU

Tích hợp AXI4-Lite, AXI4-Full, AXI4-Stream và ngoại vi UART

Triển khai trên FPGA Artix-7 bằng Vivado 2025.1

PHẠM VI BÁO CÁO

Kiến trúc RTL, chức năng từng khối và từng tệp chính, cơ chế dịch địa chỉ và bảo vệ truy cập DMA, ba kiểu truyền dữ liệu, ba chế độ chiếm bus, firmware PicoRV32, kiểm chứng XSim, thông lượng mô phỏng và kết quả post-route.

BÁO CÁO KỸ THUẬT

Phiên bản tổng hợp từ mã nguồn và báo cáo ngày 22/07/2026

TÓM TẮT

Đề tài xây dựng một hệ thống trên chip tối giản gồm bộ xử lý PicoRV32, bộ điều khiển DMA, IOMMU dành riêng cho DMA, bộ nhớ AXI và UART. CPU chịu trách nhiệm chạy firmware và cấu hình thanh ghi; DMA thực hiện di chuyển dữ liệu độc lập với CPU; IOMMU dịch địa chỉ ảo của DMA sang địa chỉ vật lý và kiểm tra quyền trước khi DMA được phép phát giao dịch bộ nhớ. Ba chuẩn giao tiếp được dùng đúng theo vai trò: AXI4-Lite cho cấu hình, AXI4-Full cho dữ liệu bộ nhớ và AXI4-Stream cho luồng dữ liệu UART.

DMA hỗ trợ ba hướng truyền: bộ nhớ sang bộ nhớ, ngoại vi sang bộ nhớ và bộ nhớ sang ngoại vi. Mỗi lệnh có thể chạy theo Burst, Cycle-Stealing hoặc Transparent. IOMMU dùng bảng trang do phần mềm lập trình gồm 16 mục, TLB 4 mục fully associative, trang 4 KiB, kiểm tra quyền đọc/ghi và thay thế pseudo-LRU. Mã RTL được kiểm tra bằng testbench tự đối chiếu dữ liệu, sau đó tổng hợp và triển khai trên Artix-7 XC7A35T.

KẾT QUẢ CHÍNH

Toàn bộ các bài test chức năng đều PASS. Thiết kế post-route đáp ứng clock 149,992 MHz (chu kỳ 6,667 ns), WNS = +0,328 ns, không có endpoint vi phạm setup/hold. Tài nguyên sử dụng: 4.038 LUT, 3.557 FF, 16 BRAM, 0 DSP và 16 I/O.

Từ khóa

DMA; IOMMU; TLB; pseudo-LRU; PicoRV32; AXI4-Lite; AXI4-Full; AXI4-Stream; UART; FPGA; Vivado.

Cấu trúc báo cáo

1. Chương 1 - Mục tiêu, phạm vi và yêu cầu thiết kế
2. Chương 2 - Kiến trúc tổng thể của hệ thống
3. Chương 3 - Kiến trúc và cơ chế hoạt động của DMA
4. Chương 4 - Kiến trúc và cơ chế hoạt động của DMA-side IOMMU
5. Chương 5 - CPU PicoRV32, firmware, UART và hệ thống bus AXI
6. Chương 6 - Chức năng các tệp mã nguồn chính
7. Chương 7 - Phương pháp và kết quả kiểm chứng chức năng
8. Chương 8 - Kết quả tổng hợp, implementation và hiệu năng
9. Chương 9 - Đánh giá, giới hạn và hướng phát triển
10. Phụ lục - Bản đồ thanh ghi, lệnh chạy và vị trí báo cáo

Danh mục từ viết tắt

Thuật ngữ	Ý nghĩa
DMA	Direct Memory Access - truyền dữ liệu trực tiếp không cần CPU chép từng word
IOMMU	Input/Output Memory Management Unit - dịch và bảo vệ truy cập của thiết bị bus-master
TLB	Translation Lookaside Buffer - bộ nhớ đệm ánh xạ địa chỉ
VA / PA	Virtual Address / Physical Address - địa chỉ ảo / địa chỉ vật lý
VPN / PPN	Virtual Page Number / Physical Page Number
FSM	Finite State Machine - máy trạng thái hữu hạn
AXI	Advanced eXtensible Interface
M2M / S2M / M2S	Memory-to-Memory / Stream-to-Memory / Memory-to-Stream

1. MỤC TIÊU, PHẠM VI VÀ YÊU CẦU THIẾT KẾ

1.1. Mục tiêu

Mục tiêu của đề án là tạo một IP DMA có thể tích hợp vào SoC, hỗ trợ nhiều hướng truyền và chính sách sử dụng bus, đồng thời buộc mọi truy cập bộ nhớ do DMA phát ra phải được IOMMU cấp phép. Hệ thống cuối cùng phải có CPU thật để cấu hình, có bộ nhớ, có ngoại vi UART, dùng giao tiếp AXI rõ ràng và có kết quả mô phỏng lần triển khai FPGA.

- 11.DMA và IOMMU là hai khối chức năng riêng, kết nối bằng giao thức yêu cầu/đáp ứng nội bộ; chúng chỉ dùng chung ngân hàng thanh ghi AXI4-Lite ở mức IP tích hợp.
- 12.CPU không nằm trên đường dữ liệu DMA. CPU chỉ lập trình descriptor, bảng trang, khởi động lệnh và đọc trạng thái/ngắt.
- 13.IOMMU bảo vệ truy cập của DMA; nó không dịch địa chỉ lệnh hoặc dữ liệu của PicoRV32.
- 14.Thiết bị ngoại vi duy nhất là UART; đường DMA-UART dùng AXI4-Stream.
- 15.Thiết kế phải tổng hợp được, implementation thành công và đáp ứng clock mục tiêu xấp xỉ 150 MHz.

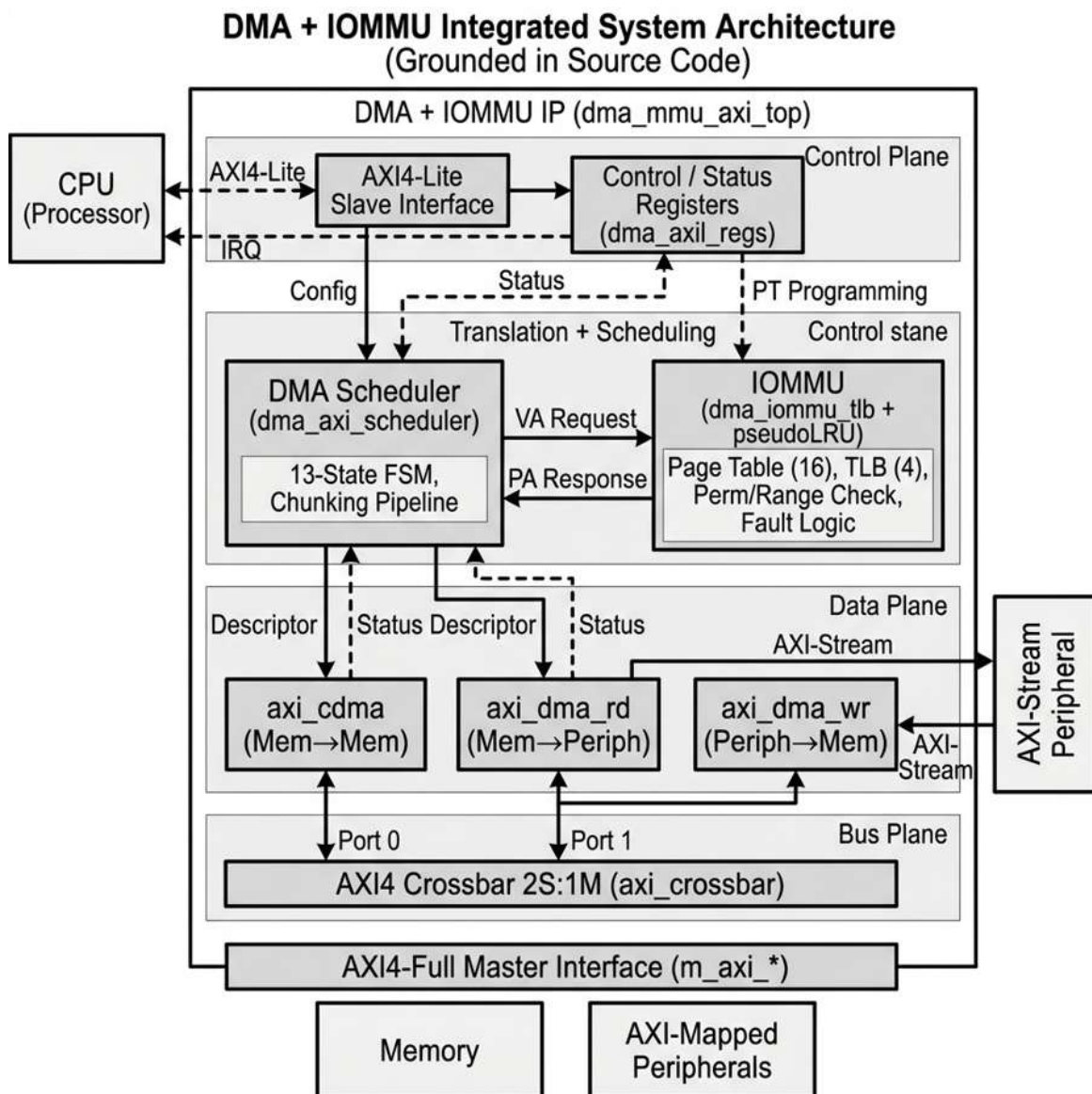
1.2. Phạm vi kiểm chứng

Bộ kiểm chứng gồm hai lớp. Lớp IP kiểm tra trực tiếp DMA/IOMMU/AXI với đầy đủ ba hướng truyền, ba chế độ bus và lỗi quyền. Lớp SoC chạy firmware thật trên PicoRV32 để lập trình IOMMU, khởi động DMA M2M, kiểm tra dữ liệu và phát ký tự kết quả qua UART. Testbench thống nhất chạy hai lớp này song song và chỉ báo PASS khi cả hai cùng hoàn thành.

ĐIỂM CẦN HIỂU ĐÚNG

Test đầy đủ S2M và M2S sử dụng mô hình AXI-Stream ngoại vi trong testbench. Test SoC chạy firmware hiện tại xác nhận CPU + IOMMU + DMA cho M2M và UART báo trạng thái. Vì vậy không nên mô tả rằng firmware PicoRV32 hiện đã chạy cả ba chế độ qua chân serial UART.

2. KIẾN TRÚC TỔNG THỂ CỦA HỆ THỐNG



Hình 2.1. Kiến trúc tích hợp DMA + IOMMU theo các khối RTL chính.

2.1. Phân lớp kiến trúc

Kiến trúc được chia thành ba mặt phẳng. Control plane chứa CPU, bộ giải mã địa chỉ AXI4-Lite và ngân hàng thanh ghi. Translation/scheduling plane chứa scheduler và IOMMU. Data plane chứa ba data mover, AXI crossbar, RAM và AXI-Stream UART. Cách tách này giúp đường điều khiển không mang payload và cho phép DMA truyền dữ liệu sau khi CPU đã phát lệnh.

Mặt phẳng	Thành phần	Vai trò
Điều khiển	PicoRV32, router, dma_axil_regs	Lập trình descriptor, PT/TLB, đọc trạng thái, nhận IRQ
Dịch/lập lịch	dma_axi_scheduler, dma_iommu_tlb	Chia lệnh thành chunk, xin dịch VA→PA, kiểm tra quyền, chọn engine
Dữ liệu	axi_cdma, axi_dma_rd, axi_dma_wr, crossbar, RAM, UART	Thực hiện handshake và vận chuyển payload

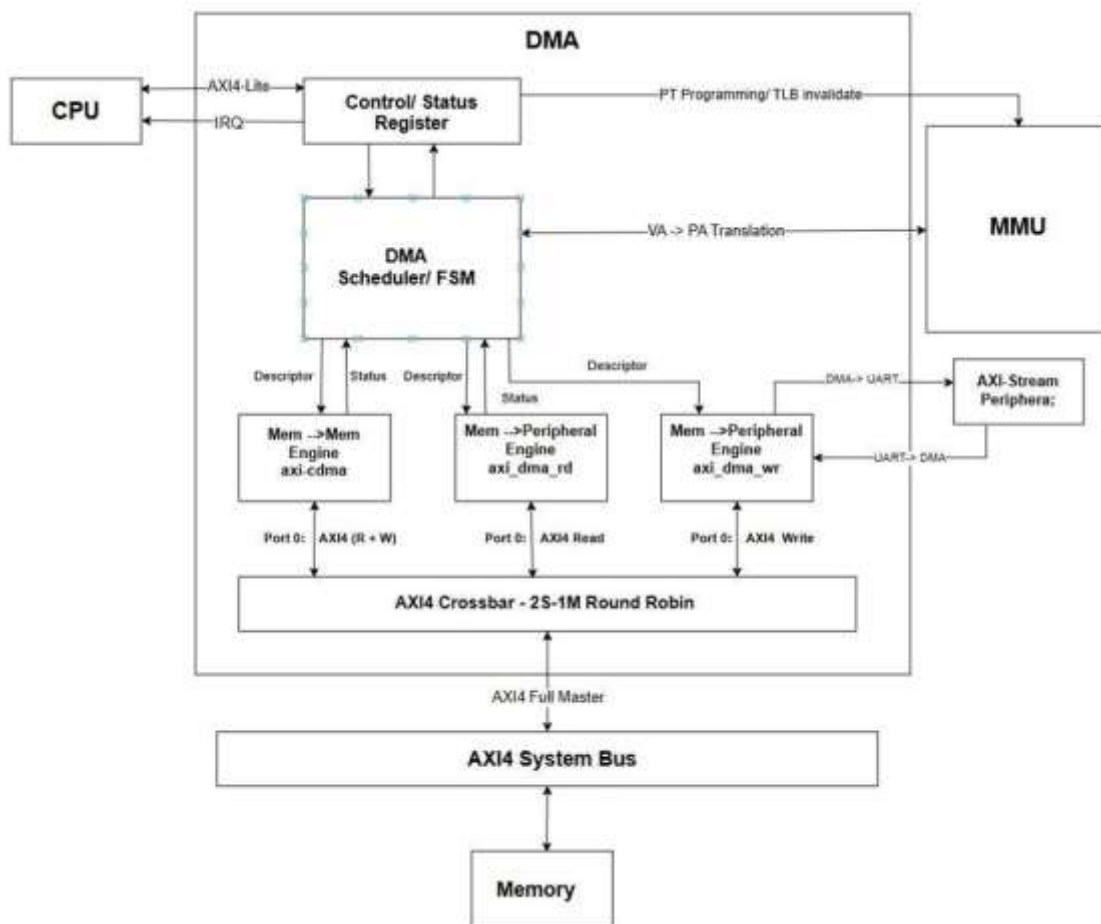
2.2. Luồng điều khiển

- 16.Firmware chạy trên PicoRV32 ghi bảng trạng của IOMMU qua vùng thanh ghi 0x1000_0000.
- 17.CPU ghi VA nguồn, VA đích, độ dài, hướng truyền, chế độ DMA và kích thước burst.
- 18.CPU ghi bit START. Scheduler chốt cấu hình và bắt đầu xử lý lệnh.
- 19.Scheduler gửi VA, length và loại truy cập R/W tới IOMMU; chỉ khi nhận allow và PA hợp lệ mới cấp descriptor cho data mover.
- 20.Data mover phát giao dịch AXI4-Full hoặc AXI4-Stream. Khi hoàn tất, scheduler cập nhật done/fault; ngân hàng thanh ghi giữ trạng thái và phát IRQ nếu được bật.

2.3. Luồng dữ liệu

Hướng	Đường dữ liệu	Engine
Memory → Memory	RAM --AXI Read→ buffer --AXI Write→ RAM	axi_cdma
Peripheral → Memory	UART/AXIS --AXI-Stream→ AXI Write → RAM	axi_dma_wr
Memory → Peripheral	RAM --AXI Read→ AXI-Stream → UART	axi_dma_rd

3. KIẾN TRÚC VÀ CƠ CHẾ HOẠT ĐỘNG CỦA DMA



Hình 3.1. Sơ đồ DMA do người thiết kế cung cấp. Khi đọc sơ đồ, khối `axi_dma_wr` phải hiểu là `Peripheral/Stream → Memory`.

3.1. Khối thanh ghi điều khiển và trạng thái

`dma_axil_regs` là AXI4-Lite slave của IP. Khối này tách kênh AW và W đúng quy tắc AXI4-Lite, hỗ trợ WSTRB, lưu descriptor DMA, tạo xung START, giữ cờ done/fault theo kiểu sticky, tạo IRQ và cung cấp cổng lập trình bảng trạng. Việc DMA và IOMMU cùng xuất hiện trong một bank thanh ghi không làm chúng trở thành một khối logic duy nhất; đây chỉ là giao diện cấu hình chung cho CPU.

3.2. DMA Scheduler/FSM

`dma_axi_scheduler` là bộ điều phối trung tâm gồm 13 trạng thái: IDLE, PREPARE, giới hạn theo trang nguồn/đích, yêu cầu và chờ IOMMU cho nguồn/đích, phát descriptor, chờ engine, GAP, DONE và FAULT. Scheduler giữ VA hiện tại, PA đã dịch, số byte còn lại và độ dài chunk. Chunk luôn bị giới hạn bởi số byte còn lại, kích thước burst tối đa và biên trang 4 KiB.

```

type    = 0:    Memory    ->    Memory                mode    = 0:    Burst
type    = 1:    Peripheral ->    Memory                mode    = 1:    Cycle-Stealing
type    = 2:    Memory    ->    Peripheral            mode    = 2:    Transparent
chunk <= min(remaining, burst_limit, bytes_to_page_boundary)

```

3.3. Ba hướng truyền

Memory-to-Memory sử dụng axi_cdma. Engine nhận PA nguồn, PA đích và chiều dài, phát burst đọc vào FIFO rồi phát burst ghi. Đây là đường có khả năng tận dụng AXI tốt nhất vì cả nguồn và đích đều là bộ nhớ.

Peripheral-to-Memory sử dụng axi_dma_wr. Dữ liệu từ UART hoặc mô hình ngoại vi đi vào qua S_AXIS; engine đóng gói thành các beat AXI Write và ghi vào RAM.

Memory-to-Peripheral sử dụng axi_dma_rd. Engine đọc RAM bằng AXI Read, đẩy dữ liệu ra M_AXIS và tạo TLAST ở beat cuối của lệnh.

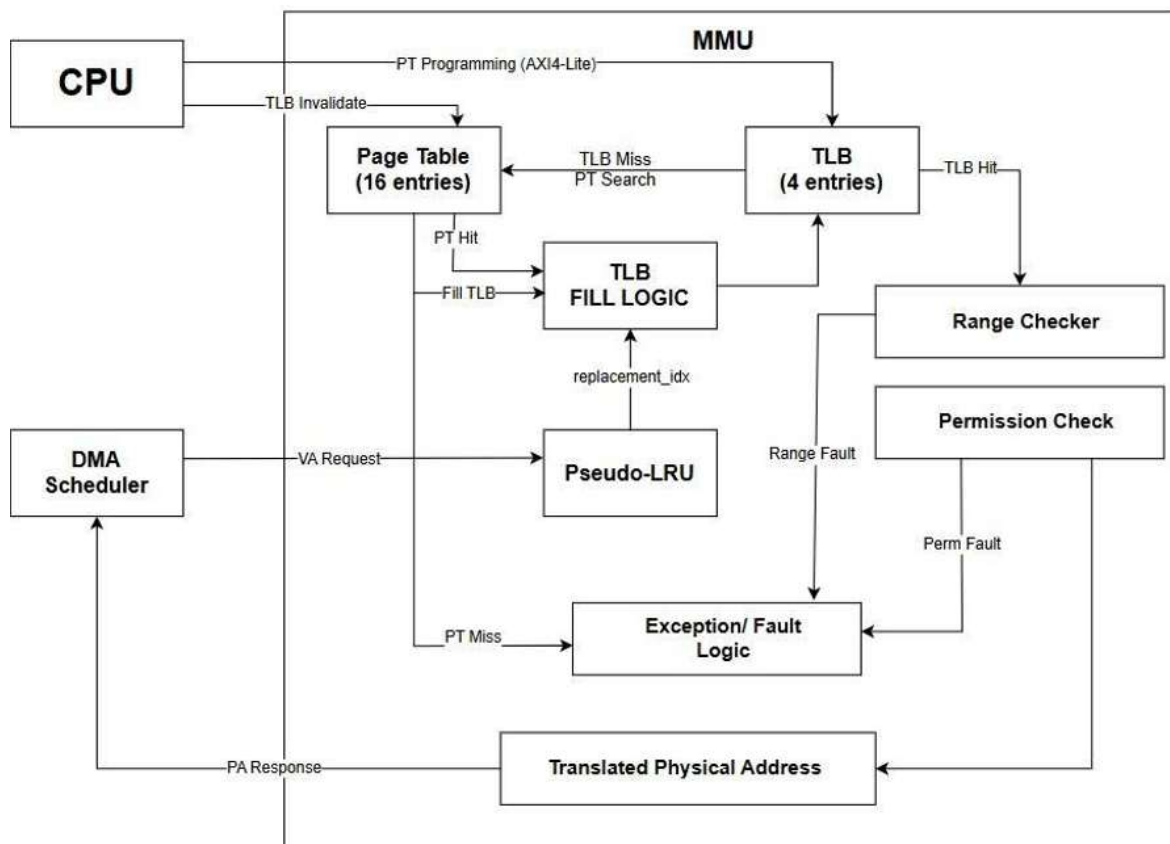
3.4. Ba chế độ sử dụng bus

Chế độ	Cách thực hiện trong RTL	Tác động
Burst	Mỗi descriptor chứa nhiều beat; bị chặn bởi cfg_burst_words, AXI max và biên trang	Thông lượng cao, chiếm bus liên tục trong burst
Cycle-Stealing	Mỗi chunk đúng 1 beat, sau khi engine xong đi qua ST_GAP	Nhường bus giữa các beat; thông lượng giảm có chủ đích
Transparent	Mỗi chunk 1 beat; chỉ phát khi cpu_bus_idle_i = 1	Không tranh bus khi CPU bận; độ trễ phụ thuộc thời gian bus rảnh

3.5. Các data mover và bộ đệm

- 21.axi_cdma: có FSM đọc, FSM ghi, FIFO dữ liệu và status; dùng cho M2M.
- 22.axi_dma_rd: chuyển AXI memory-mapped read thành AXI-Stream; dùng cho M2S.
- 23.axi_dma_wr: chuyển AXI-Stream thành AXI memory-mapped write; dùng cho S2M.
- 24.axi_crossbar nội bộ: ghép port CDMA và port streaming vào một AXI4-Full master; trọng tài đọc và ghi độc lập theo round-robin.

4. KIẾN TRÚC VÀ CƠ CHẾ HOẠT ĐỘNG CỦA DMA-SIDE IOMMU



Hình 4.1. Kiến trúc IOMMU phục vụ DMA: bảng trang phần mềm, TLB, pseudo-LRU, kiểm tra quyền/range và phản hồi PA.

4.1. Phân biệt MMU và IOMMU

Khối trong đồ án là DMA-side IOMMU. Nó nhận yêu cầu từ DMA scheduler, không nhận instruction VA hoặc data VA của CPU. PicoRV32 vẫn truy cập RAM bằng địa chỉ vật lý qua router. Tên “MMU” trong một số hình mang ý nghĩa khái quát; tên chính xác theo RTL là `dma_iommu_tlb`.

4.2. Bảng trang phần mềm và TLB

Cấu trúc	Quy mô	Tổ chức	Nội dung mỗi mục
Page Table	16 entries	Fully associative, do CPU ghi	VPN, PPN, Valid, Read, Write
TLB	4 entries	Fully associative cache	VPN, PPN, Valid, Read, Write

Với AXI address 16 bit và `PAGE_SHIFT = 12`, mỗi trang có kích thước 4 KiB; VPN và PPN rộng 4 bit. Bảng 16 mục đủ biểu diễn toàn bộ 16 trang của không gian 64 KiB trong cấu hình hiện tại. “Fully associative”

nghĩa là VPN yêu cầu được so sánh song song với tất cả mục hợp lệ, không bị cố định vào một set hoặc index.

4.3. Pipeline tra cứu ba trạng thái

25.LOOKUP_IDLE: nhận và giữ req_vaddr, req_len và req_write khi ready/valid bắt tay.

26.LOOKUP_MATCH: so sánh song song TLB và page table, tính end address, kiểm tra overflow và yêu cầu không vượt biên một trang.

27.LOOKUP_RESP: chọn PPN/quyền đã đăng ký, tạo PA, trả allow hoặc fault; nếu TLB miss nhưng PT hit thì đồng thời fill TLB.

Việc tách compare và selected-entry mux qua hai chu kỳ là tối ưu timing quan trọng: đường VPN compare → priority → PPN mux không còn nằm trọn trong một chu kỳ dài.

4.4. Pseudo-LRU và ưu tiên entry invalid

Khi cần nạp TLB, logic trước tiên tìm một entry invalid. Chỉ khi cả bốn entry đều valid mới sử dụng chỉ số nạp nhân từ pseudoLRU.sv. Pseudo-LRU lưu lịch sử sử dụng theo cây bit để xấp xỉ Least Recently Used với ít trạng thái hơn LRU chính xác. Mỗi TLB hit hoặc PT hit được fill đều cập nhật cây tại LOOKUP_RESP.

4.5. Kiểm tra range, permission và fault

Fault nội bộ	Mã	Điều kiện
NONE	0	Ánh xạ hợp lệ và quyền phù hợp
PAGE	1	TLB miss và không tìm thấy VPN hợp lệ trong page table
PERMISSION	2	Trang tồn tại nhưng không cho phép đọc hoặc ghi yêu cầu
RANGE	3	Length = 0, tràn địa chỉ hoặc yêu cầu vượt biên trang 4 KiB

Scheduler bổ sung ngữ cảnh nguồn/đích vào mã lỗi. Ví dụ 0x32 trong test nghĩa là base 0x30 của IOMMU đích cộng với fault permission = 2: DMA muốn ghi vào trang chỉ đọc nên bị từ chối trước khi phát AXI Write.

5. CPU PicoRV32, FIRMWARE, UART VÀ HỆ THỐNG BUS AXI

5.1. Vai trò của PicoRV32

PicoRV32 là bộ xử lý điều khiển 32 bit RISC-V. Trong hệ thống này CPU thực thi firmware từ RAM, khởi tạo dữ liệu, lập trình bảng trang IOMMU, cấu hình DMA, polling trạng thái và phát ký tự qua UART. CPU không tự chép toàn bộ payload sau khi DMA đã chạy. Phiên bản được instantiate là picorv32_axi; các wrapper Wishbone không tham gia hierarchy hiện hành.

Cấu hình CPU	Giá trị/ý nghĩa
ISA và bus	RV32I, AXI4-Lite master; có nhân PCPI
Thanh ghi	x0-x31, dual-port register file
Timing	TWO_CYCLE_COMPARE = 1, TWO_CYCLE_ALU = 1 để rút ngắn critical path
IRQ	DMA ở bit 5; UART ở bit 4
Boot	Reset vector 0x0000_0000; stack 0x0000_FFFC

5.2. Firmware demo

soc_demo.S là mã Assembly nguồn. Nó gửi ký tự S, ghi bốn word 0x11111111...0x44444444 vào RAM tại 0x4000, tạo hai ánh xạ identity VPN 4→PPN 4 và VPN 5→PPN 5 với quyền R/W, cấu hình lệnh M2M 16 byte từ 0x4000 sang 0x5000, chờ DONE, so sánh bốn word đích và gửi P nếu đúng hoặc F nếu sai. soc_demo.hex là mã máy được nạp vào RAM bằng \$readmemh; soc_map.h là bản đồ địa chỉ dành cho firmware C/C++ tương lai.

UART	'S'	->	program	PT[4],	PT[5]	->	initialize	RAM
SRC=0x4000,			DST=0x5000,		LENGTH=16,		CONFIG=0x00000400	
START=1	->	poll	STATUS.done	->	compare 4 words	->	UART	'P'/'F'

5.3. Bản đồ địa chỉ CPU

Khoảng địa chỉ	Đích	Giao tiếp
0x0000_0000 - 0x0000_FFFF	AXI RAM 64 KiB	AXI4-Lite từ CPU; AXI4-Full từ DMA
0x1000_0000 - 0x1000_00FF	Thanh ghi DMA/IOMMU	AXI4-Lite
0x2000_0000 - 0x2000_00FF	Thanh ghi UART	AXI4-Lite

5.4. Ba loại AXI trong đồ án

Giao tiếp	Nối giữa	Tác dụng
AXI4-Lite	PicoRV32 $\leftrightarrow$ router $\leftrightarrow$ RAM/DMA regs/UART regs	Đọc lệnh, dữ liệu CPU và cấu hình/status; không burst
AXI4-Full	DMA engines $\leftrightarrow$ crossbar $\leftrightarrow$ AXI RAM	Mang payload bộ nhớ, hỗ trợ burst, ID và response
AXI4-Stream	axi_dma_rd/axi_dma_wr $\leftrightarrow$ UART	Luồng payload không địa chỉ, valid/ready, keep, last

5.5. UART

uart_axil_axis kết hợp thành ghi CPU với bộ chuyển đổi AXI-Stream 32 bit. Ở chiều DMA $\rightarrow$ UART, một word và TKEEP được tách thành byte serial. Ở chiều UART $\rightarrow$ DMA, các byte nhận được ghép thành word và phát ra M_AXIS; phần word chưa đủ bốn byte có thể flush. simpleuart_dma thực hiện truyền/nhận 8N1, bộ chia baud, đồng bộ RX và phát hiện overrun.

6. CHỨC NĂNG CÁC TẬP MÃ NGUỒN CHÍNH

6.1. Tập RTL cấp hệ thống và IP

Tập	Chức năng chính	Quan hệ
dma_mmu_picorv32_soc.sv	Top tổng thể CPU + router + DMA/IOMMU + UART + system crossbar + RAMS	Top synthesis hiện tại
picorv32_axil_router.sv	Giải mã ba vùng địa chỉ; đếm AW/W độc lập; sinh cpu_bus_idle	CPU → RAM/DMA/UART
dma_mmu_axi_top.sv	Top IP DMA/IOMMU; AXI-Lite slave, AXI-Full master, AXI-Stream source/sink	Được instantiate trong SoC
dma_axil_regs.sv	Bank thanh ghi DMA/IOMMU, sticky status, IRQ, PT programming	CPU control plane
dma_axi_scheduler.sv	FSM 13 trạng thái, chia chunk, gọi IOMMU, chọn engine, tạo done/fault	Điều phối toàn bộ DMA
dma_iommu_tlb.sv	PT 16, TLB 4, 3-stage lookup, permission/range/fault	Cấp PA/allow cho scheduler
uart_axil_axis.sv	UART register bank và adapter AXI-Stream 32-bit	CPU + DMA ↔ UART
simpleuart_dma.sv	Byte engine UART 8N1, divider, RX synchronizer	Được bọc bởi uart_axil_axis

6.2. Tập lõi AXI và thuật toán hỗ trợ

Tập	Vai trò
axi_cdma.v	Engine M2M: AXI Read + FIFO + AXI Write, descriptor/status.
axi_dma_rd.v	Engine Memory→Stream: AXI Read thành M_AXIS, quản lý TLAST/TKEEP.
axi_dma_wr.v	Engine Stream→Memory: nhận S_AXIS và phát AXI Write.
axi_crossbar.v	Ghép nhiều AXI slave-side ports vào master-side port; định tuyến ID/response.
arbiter.v	Trọng tài round-robin có fairness; kênh đọc/ghi độc lập trong crossbar.
priority_encoder.v	Chọn request theo thứ tự ưu tiên do arbiter xác định.
axi_ram.v	Mô hình/khối RAM AXI dùng chung cho firmware và dữ liệu.
pseudoLRU.sv	Cây pseudo-LRU chọn TLB victim khi không còn entry invalid.
picorv32.v	Chứa core PicoRV32, picorv32_axi, AXI adapter và PCPI multiplier được dùng.

6.3. Firmware, testbench, constraint và script

Tệp	Mục đích
firmware/soc_demo.S	Assembly nguồn chạy thật trên CPU; tạo input, cấu hình và kiểm tra M2M.
firmware/soc_demo.hex	Mã máy khởi tạo RAM; là đầu vào chương trình của PicoRV32.
firmware/soc_map.h	Macro địa chỉ/thanh ghi cho firmware C/C++; không tự chạy trong demo Assembly.
tb/tb_dma_mmu_axi_top.sv	Self-check đầy đủ 3 hướng, 3 mode, IOMMU fault, data I/O và throughput.
tb/tb_dma_mmu_picorv32_soc.sv	Chạy firmware PicoRV32, theo dõi UART và kiểm tra dữ liệu M2M.
tb/tb_dma_iommu_picorv32_unified.sv	Top simulation thống nhất; chờ hai testbench con cùng PASS.
constraints/dma_mmu_picorv32_soc.xdc	Clock 6,667 ns và các constraint timing của top SoC.
scripts/generate_reports.tcl	Xuất timing, utilization và các báo cáo sau run.

7. PHƯƠNG PHÁP VÀ KẾT QUẢ KIỂM CHỨNG CHỨC NĂNG

7.1. Nguyên tắc self-checking

Testbench không chỉ nhìn waveform mà tự so sánh input và output. Mỗi word được kiểm tra tại đúng địa chỉ/beat; sai dữ liệu, sai TLAST, sai số burst, không có khoảng nhường bus hoặc IOMMU cho phép truy cập trái quyền đều làm tăng errors và kết thúc FAIL. File log vì vậy là kết quả do mô phỏng XSim tạo ra, không phải chuỗi PASS được viết tay không điều kiện.

7.2. Kết quả test IP DMA/IOMMU/AXI

Test	Input	Output/điều kiện kiểm	Kết quả
M2M / Burst	8 word A5000000...A5000007 tại VA 0x1000	8 word trùng khớp tại VA 0x2000; AR=1, AW=1, ARLEN=AWLEN=7	PASS
S2M / Cycle-Stealing	8 beat S_AXIS D1000000...D1000007	RAM 0x3000...0x301C khớp; 8 giao dịch write 1 beat và có release gap	PASS
M2S / Transparent	8 word A5000000...A5000007 tại VA 0x1000	8 beat M_AXIS khớp; TLAST ở beat 8; chờ CPU bận 29 chu kỳ	PASS
IOMMU permission	S2M write tới VA 0x4000 trên page chỉ đọc	allow=0; fault=0x32; không ghi sai vào RAM	PASS

Thống kê cuối test hiện tại là TLB hit = 1, miss = 4. Số hit thấp hơn các phiên bản trước vì scheduler tái sử dụng địa chỉ đã dịch trong một trang và chỉ gọi IOMMU khi cần; đây là tối ưu hợp lệ, không phải lỗi TLB.

7.3. Kết quả test PicoRV32 SoC

Quan sát	Giá trị
UART bắt đầu	0x53 ('S') tại 690.000 ps
Dữ liệu nguồn	0x11111111, 0x22222222, 0x33333333, 0x44444444 tại PA 0x4000
Dữ liệu đích	Bốn word giống nguồn tại PA 0x5000
UART kết thúc	0x50 ('P') tại 6.531.000 ps
Kết luận	CPU đã cấu hình IOMMU và DMA; M2M copy verified; SOC TEST PASSED

7.4. Ý nghĩa của test thống nhất

unified_verification.log ghi DMA/IOMMU AXI component PASSED và PicoRV32 CPU/SoC component PASSED tại 6.532.000 ps, sau đó mới ghi ALL UNIFIED TESTS PASSED. Hai testbench con chạy song song để rút ngắn thời gian và gom kết quả vào một project Vivado duy nhất.

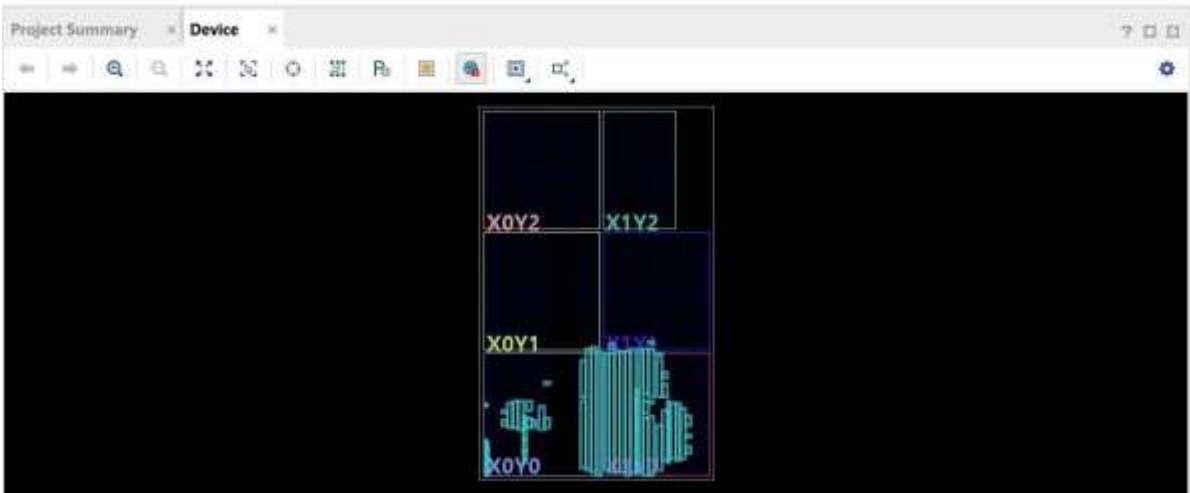
TÍNH TRUNG THỰC CỦA KẾT QUẢ

Các dòng dữ liệu MATCH và PASS đến từ điều kiện so sánh trong testbench. Riêng mục DMA standalone trong báo cáo throughput là tham chiếu lý thuyết 1 word/clock, không phải một instance DMA độc lập đã được mô phỏng. Báo cáo này luôn ghi rõ sự khác biệt đó.

8. KẾT QUẢ SYNTHESIS, IMPLEMENTATION VÀ HIỆU NĂNG

8.1. Điều kiện triển khai

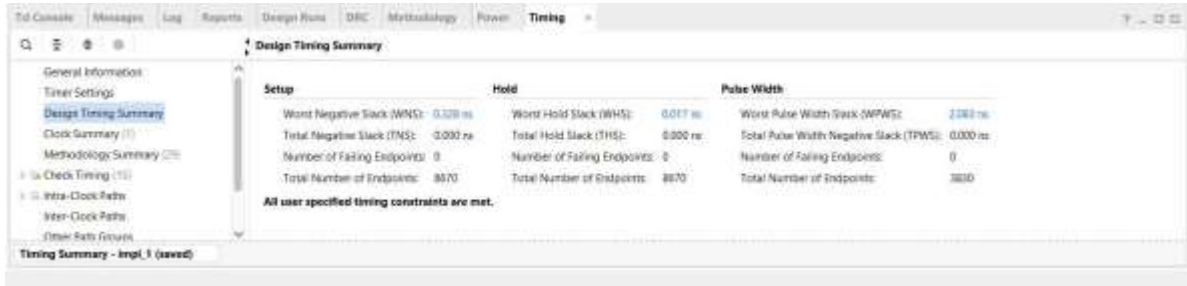
Thông số	Giá trị
Công cụ	AMD Vivado 2025.1
FPGA	Artix-7 XC7A35T, package CPG236, speed grade -1
Top	dma_mmu_picorv32_soc
Clock constraint	6,667 ns = 149,992 MHz
Data width	32 bit; AXI address width 16 bit
Page	4 KiB; PT 16 entries; TLB 4 entries



Hình 8.1. Phân bố logic sau implementation trên Artix-7 XC7A35T.

Ảnh placement cho thấy phần lớn logic và RAM được đặt trong các vùng phía dưới thiết bị. Đây là kết quả hợp lệ của placer; mật độ không đồng đều không tự động đồng nghĩa với lỗi. Đánh giá chất lượng route phải dựa trên timing, congestion và DRC.

8.2. Timing post-route



Hình 8.2. Design Timing Summary: tất cả constraint timing đã đạt.



Hình 8.3. Clock Summary: sys_clk 6,667 ns, tương đương 149,992 MHz.

Chỉ số	Kết quả	Diễn giải
WNS	+0,328 ns	Setup còn dư 0,328 ns; không vi phạm
TNS	0,000 ns	Không có tổng slack âm
Setup failing endpoints	0 / 8.670	Tất cả endpoint setup đạt
WHS / THS	+0,017 ns / 0,000 ns	Không vi phạm hold
WPWS / TPWS	+2,083 ns / 0,000 ns	Không vi phạm pulse width
Fmax ước lượng toàn hệ	157,754 MHz	$6,667 - 0,328 = 6,339$ ns; là ước lượng theo slack

CÁCH BÁO CÁO FMAX ĐÚNG

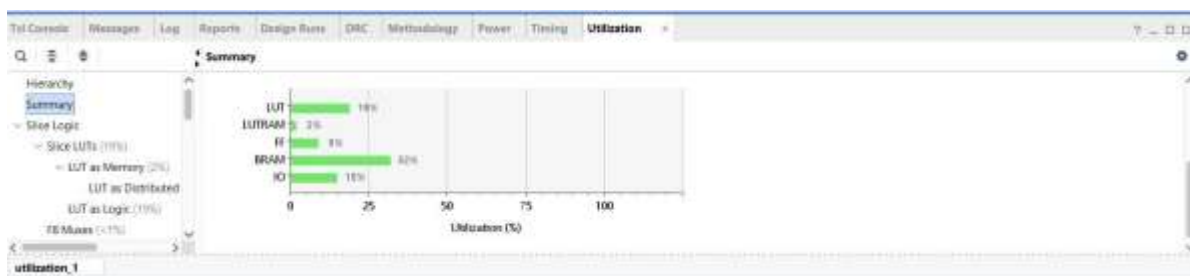
149,992 MHz là tần số đã được constrain và implementation chứng minh đạt. 157,754 MHz là trần ước lượng từ critical period hiện tại, không phải tần số đã chạy lại route và xác nhận. Khi bảo vệ đồ án, nên công bố 150 MHz verified và nêu 157,754 MHz như headroom ước lượng.

8.3. Tần số đường nội bộ DMA và IOMMU

Khối	Worst slack	Critical period	Fmax ước lượng	Logic levels
DMA core	+0,328 ns	6,339 ns	157,754 MHz	7
IOMMU core	+0,918 ns	5,749 ns	173,943 MHz	10

Hai số trên được trích từ đường register-to-register sau route trong bối cảnh physical của full SoC. Chúng không phải thông lượng DMA và cũng không phải kết quả route độc lập của từng khối.

8.4. Tài nguyên



Hình 8.4. Tổng quan tỷ lệ sử dụng tài nguyên sau implementation.

Tài nguyên	Đã dùng	Có sẵn	Tỷ lệ
Slice LUT	4.038	20.800	19,41%
Slice Register	3.557	41.600	8,55%
Block RAM Tile	16	50	32,00%
DSP	0	90	0,00%
Bonded I/O	16	106	15,09%

BRAM là hạng mục có tỷ lệ cao nhất do AXI RAM chứa firmware và dữ liệu. Thiết kế không dùng DSP; phép nhân của cấu hình hiện tại được ánh xạ không làm phát sinh DSP trong báo cáo cuối. Mức LUT và FF còn dư đủ cho việc mở rộng thêm thanh ghi, descriptor queue hoặc bộ đếm hiệu năng.

8.5. Công suất ước lượng

Báo cáo power post-route của project thống nhất ước lượng tổng công suất on-chip 0,135 W, gồm 0,064 W dynamic và 0,071 W device static ở nhiệt độ junction 25,7 °C. Đây là ước lượng dựa trên activity mặc định/khai báo của Vivado; muốn số sát phần cứng cần dùng SAIF từ mô phỏng và điều kiện board thực tế.

8.6. Thông lượng mô phỏng

Bài test	Mốc DMA lý thuyết	Start→Done	Data window	AXI full transaction
M2M / Burst	599,970 MB/s	119,994 MB/s (40 chu kỳ)	399,980 MB/s	Read 436,342; Write 266,653 MB/s
S2M / Cycle-Stealing	599,970 MB/s	35,819 MB/s (134 chu kỳ)	41,024 MB/s	Write 41,377 MB/s
M2S / Transparent	599,970 MB/s	40,334 MB/s (119 chu kỳ)	47,056 MB/s	Read 47,056 MB/s

Mốc 599,970 MB/s được tính từ $32 \text{ bit} \times 149,993 \text{ MHz}$ và giả định một word mỗi chu kỳ, không có wait. Trong M2M Burst, các beat dữ liệu thực sự đạt 100% utilization và 599,970 MB/s tức thời; start-to-done thấp hơn vì còn dịch địa chỉ, phát descriptor, latency AR/AW/B và khởi động/kết thúc engine. Cycle-Stealing và Transparent chậm hơn chủ yếu do chính sách: mỗi chunk chỉ một beat, có gap hoặc phải chờ `cpu_bus_idle`. Đây là đánh đổi thiết kế chứ không phải AXI bị hỏng.

9.4. Kết luận

Đồ án đã hình thành một SoC thử nghiệm hoàn chỉnh và nhất quán: PicoRV32 cấu hình hệ thống, DMA vận chuyển dữ liệu theo ba hướng và ba chính sách bus, IOMMU dịch/bảo vệ truy cập DMA, UART đóng vai trò ngoại vi duy nhất, còn AXI cung cấp lớp liên kết tiêu chuẩn. Kết quả mô phỏng chứng minh đúng dữ liệu và đúng hành vi điều khiển; kết quả post-route chứng minh thiết kế đạt 149,992 MHz trên XC7A35T với slack dương. Với phạm vi đồ án IP số, thiết kế hiện ở mức tốt và có nền tảng rõ ràng để mở rộng thành DMA đa kênh hoặc IOMMU có page-table walker.

PHỤ LỤC A. BẢNG ĐỒ THANH GHI DMA/IOMMU

Offset	Tên	R/W	Chức năng
0x00	CONTROL	R/W	bit0 START; bit1 clear sticky status; bit2 IRQ enable
0x04	STATUS	R	bit0 busy; bit1 done; bit2 fault; bit3 IRQ; [15:8] fault code
0x08	SRC_ADDR	R/W	Địa chỉ ảo nguồn
0x0C	DST_ADDR	R/W	Địa chỉ ảo đích
0x10	LENGTH	R/W	Số byte của lệnh
0x14	CONFIG	R/W	[1:0] type; [3:2] mode; [15:8] burst words
0x18	FAULT	R	Mã fault sticky
0x20	PT_INDEX	R/W	Chỉ số entry page table
0x24	PT_VPN	R/W	Virtual Page Number
0x28	PT_PPN	R/W	Physical Page Number
0x2C	PT_FLAGS	R/W	bit0 valid; bit1 read; bit2 write; ghi thanh ghi này commit entry
0x30	TLB_CTRL	W	bit0 invalidate toàn TLB
0x34	TLB_HITS	R	Bộ đếm TLB hit
0x38	TLB_MISSES	R	Bộ đếm TLB miss

Thanh ghi UART

Offset	Tên	R/W	Chức năng
0x00	DIVIDER	R/W	Bộ chia baud
0x04	DATA	W/R	CPU transmit / receive byte
0x08	STATUS	R	TX ready, RX valid/overrun, DMA TX/RX status
0x0C	CONTROL	R/W	DMA TX enable, DMA RX enable, flush partial RX word

PHỤ LỤC B. CÁCH MỞ PROJECT VÀ XEM KẾT QUẢ

Project thống nhất

```
C:\rtl\rtl\Project_Vivado\vivado_unified_project\DMA_IOMMU_PicoRV32_Unified.xpr
Synthesis top : dma_mmu_picorv32_soc
Simulation top: tb_dma_iommu_picorv32_unified
```

28. Mở file .xpr bằng Vivado 2025.1.
29. Chọn Run Simulation → Run Behavioral Simulation để chạy cả hai nhóm kiểm chứng.
30. Trong Tcl Console gõ run all; kiểm tra ALL UNIFIED CPU/DMA/IOMMU/AXI TESTS PASSED.
31. Chọn Run Synthesis và Run Implementation. Sau khi hoàn tất, mở Report Timing Summary và Report Utilization.

Các file kết quả quan trọng

File	Nội dung
reports/dma_mmu_axi_test.log	Input/output chi tiết, 3 hướng, 3 mode và IOMMU fault
reports/picorv32_soc_test.log	UART S/P và dữ liệu M2M do CPU cấu hình
reports/unified_verification.log	Kết luận chung của hai nhóm test
reports/dma_throughput.log	So sánh mốc DMA lý thuyết với DMA+IOMMU+AXI đo theo chu kỳ
reports/unified_timing.rpt	Timing post-route đầy đủ
reports/unified_utilization.rpt	Tài nguyên post-route
reports/unified_fmax.txt	Fmax ước lượng toàn hệ
reports/dma_iommu_separate_fmax.log	Đường timing nội bộ riêng DMA và IOMMU

Nguồn số liệu

Các thông số trong báo cáo được đối chiếu từ mã RTL trong Project_Vivado/rtl, firmware, testbench, log XSim và báo cáo Vivado post-route tại thời điểm tạo tài liệu. Hình kiến trúc và ảnh kết quả do người thiết kế cung cấp; các chỗ gọi chung là MMU được diễn giải theo module thực tế dma_iommu_tlb.